

MODULE 7 · MULTI-TENANCY & SECURITY

Kubeconfig & Contexts

The file kubectl reads to know who — and where — you are

kubectl has no identity of its own — the file supplies one

kubectl is a thin client. Every command it runs needs to know three things: **which cluster** to call, **which identity** to authenticate as, and **which namespace** to default into.

All three live in one place — the **kubeconfig** file — and a **context** is just a named bundle of the three. Switching context switches your entire working identity, instantly.

BY THE END YOU CAN

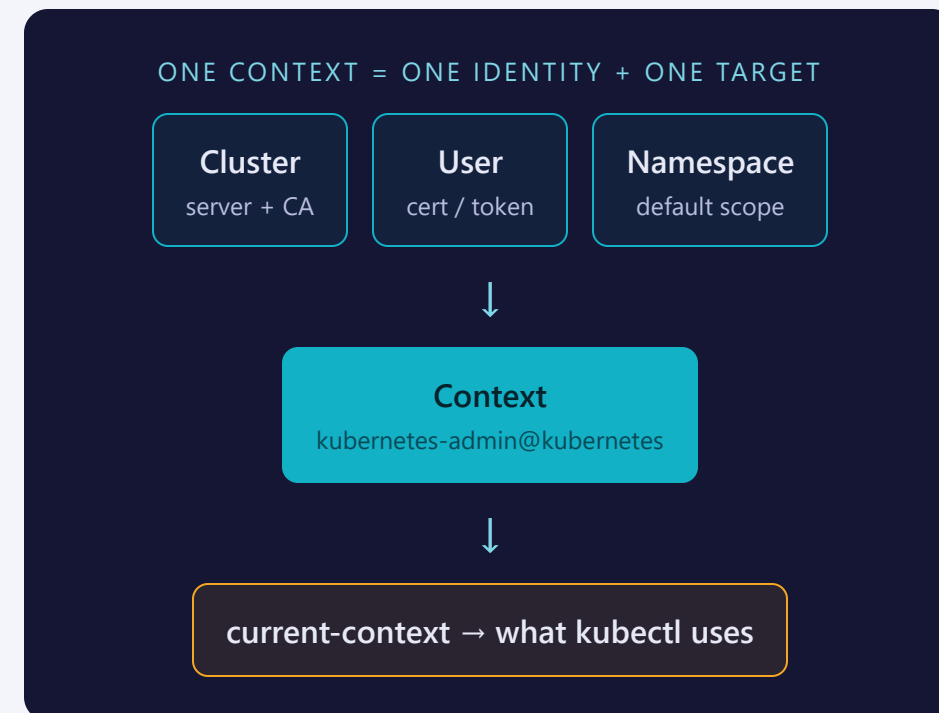
- Read a kubeconfig's three sections
- Explain what a context ties together
- Pin a namespace to a context
- Build a context from a ServiceAccount token
- Merge kubeconfig files with KUBECONFIG

CONCEPT

A context bundles cluster + user + namespace

A kubeconfig holds three parallel, named lists: **clusters** (where to talk), **users** (how to authenticate), **contexts** (which cluster+user+namespace triple is active).

A context doesn't hold credentials or endpoints itself — it just **points at one entry from each list by name**. current-context says which context kubectl uses right now.



THE THREE SECTIONS

Three lists, three jobs

SECTION	HOLDS	NAMED BY
clusters	API server URL + CA certificate	cluster name, e.g. <code>kubernetes</code>
users	How to authenticate — client cert/key, or a bearer token	user name, e.g. <code>kubernetes-admin</code>
contexts	A <code>(cluster, user, namespace)</code> triple	context name, e.g. <code>kubernetes-admin@kubernetes</code>

MENTAL MODEL

`kubectl config view` redacts cert/token data by default (DATA+OMITTED) — add `--raw` to see it. `--minify` strips everything except the current context's own cluster + user.

Just a YAML file, same shape every time

```
apiVersion: v1
kind: Config
clusters:
- name: kubernetes
  cluster:
    server: https://10.0.0.1:6443
    certificate-authority-data: DATA+OMITTED
users:
- name: kubernetes-admin
  user:
    client-certificate-data: DATA+OMITTED
    client-key-data: DATA+OMITTED
contexts:
- name: kubernetes-admin@kubernetes
  context:
    cluster: kubernetes
    user: kubernetes-admin
    namespace: default
current-context: kubernetes-admin@kubernetes
```

- **clusters / users / contexts** — three lists, each entry referenced by name
- **context.namespace** — the default namespace, part of the context itself
- **current-context** — the one name kubectl actually reads

Lives at ~/.kube/config — \$KUBECONFIG overrides the path.

Commands mutate the same file you could hand-edit

IMPERATIVE — KUBECTL CONFIG

```
kubectl config set-context --current \
  --namespace=kube-system
```

Validates flags, writes the file for you

DECLARATIVE — EDIT THE FILE

Open `~/.kube/config`, change the context's `namespace:` field

Same result — kubeconfig is plain YAML, no `apply` step

THE BRIDGE TRICK

There's no `--dry-run` here — every `kubectl config` subcommand writes straight to disk. See exactly what it wrote before you trust it:

```
$ kubectl config view --minify -o yaml # the entry a command just created
```

SEE IT

Switching context switches who you are, instantly

No re-login, no new terminal. `kubectl config use-context` swaps the active cluster+user+namespace triple for every command that follows in the same shell.

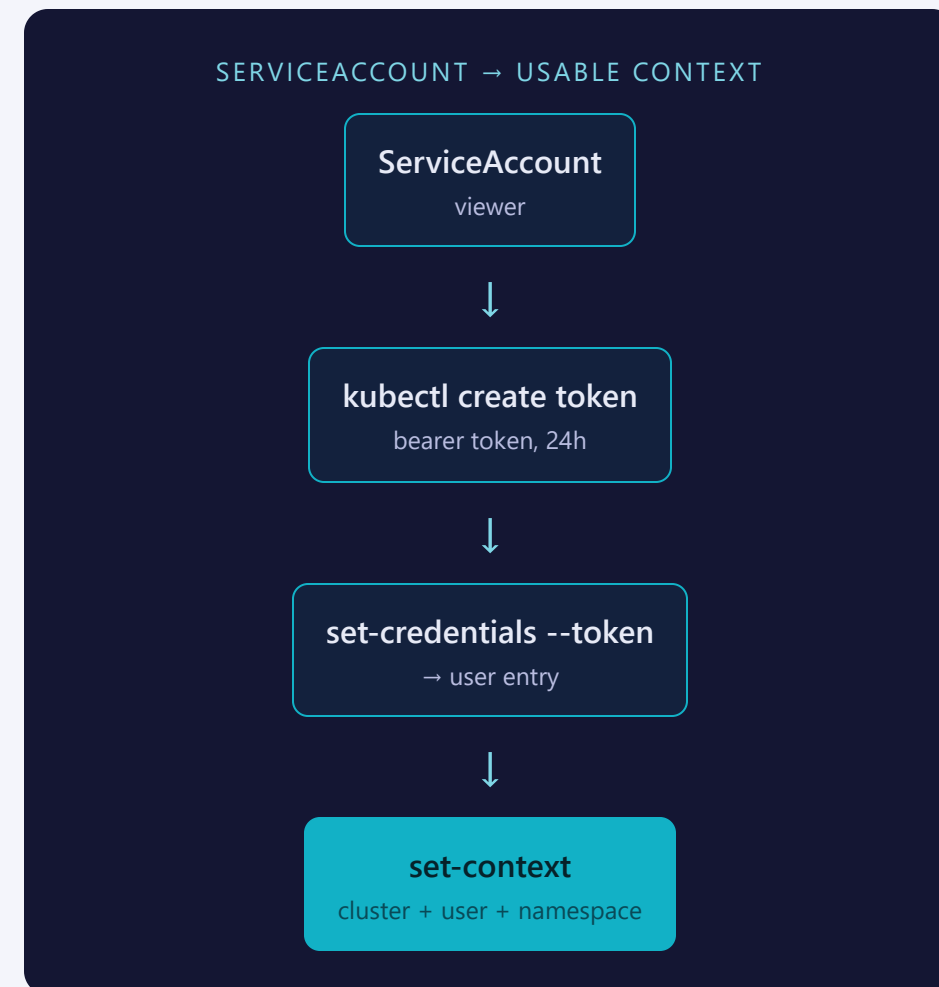
`kubectl auth whoami` proves it — same binary, same shell, a completely different identity and RBAC.

```
$ kubectl config use-context viewer-ctx
$ kubectl auth whoami
Username: system:serviceaccount:default:viewer
$ kubectl get pods
3 nginx-demo-* pods          # view allows reads
$ kubectl get secrets
Error: forbidden             # view excludes secrets
$ kubectl create deployment nginx --image=nginx
Error: forbidden             # view is read-only
```

A context doesn't need a certificate — a token will do

Any bearer token can back a user entry — including a **ServiceAccount token**, minted entirely in-cluster, no external identity provider involved.

Wire the token into a new user, point a new context at that user plus a cluster and namespace, and a full second identity is one use-context away — no second kubeconfig file needed.



Where people trip

- **Redacted ≠ missing.** `kubectl config view` shows DATA+OMITTED for real cert/token data — use `--raw` to confirm it's actually there.
- **delete-context alone leaves orphans.** It only removes the context entry — the cluster and user it pointed at stay behind. Delete all three to fully prune.
- **Namespace pinning is on the context, not the shell.** `set-context --current --namespace=X` edits the file — it persists across terminals, it isn't a session variable.
- **Merge collisions favour the first file.** With `KUBECONFIG=a:b`, duplicate cluster/user names dedupe silently to whichever appears in a first.

Commands to keep at hand

```
$ kubectl config get-contexts           # list, * marks current
$ kubectl config current-context        # which one is active
$ kubectl config use-context NAME       # switch identity + cluster
$ kubectl config set-context --current -n NS # pin a namespace
$ kubectl config set-credentials USER --token=T # build a user from a token
$ kubectl config view --minify --raw    # current context, unredacted
$ KUBECONFIG=a:b kubectl config view --flatten # merge multiple files
```

Tip: `kubectl auth whoami` is the fastest sanity check after any context switch.

RECAP

Kubeconfig & contexts in one breath

- Kubeconfig = clusters + users + contexts, plus one current-context
- A context = cluster + user + namespace, nothing more
- Any bearer token — including a ServiceAccount token — can back a context
- KUBECONFIG=a:b merges files; --flatten makes it permanent

HANDS-ON

Now run Lab 7.4
[labs/7.4/](#)

Next: [7.5 — Image Policy & OPA Gatekeeper](#)